Database Automation Using Views and Procedures

Scenario:

The e-commerce company frequently generates reports and performs repetitive database operations such as order processing, sales calculation, and customer analysis. To improve efficiency and reduce repeated SQL coding, the company wants to automate database operations using Views, Stored Procedures, and Functions. Students will implement database automation features in the existing E-Commerce Database System.

1. Customer Order Report View

Display:

- Customer Name
- Order ID
- Order Date
- Total Amount
- Order Status

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_ORDER_REPORT AS
  2  SELECT
  3      C.CUSTOMER_ID,
  4      C.FIRST_NAME,
  5      C.LAST_NAME,
  6      O.ORDER_ID,
  7      O.ORDER_DATE,
  8      O.TOTAL_AMOUNT,
  9      O.ORDER_STATUS
 10  FROM CUSTOMER C
 11  JOIN ORDERS O
 12  ON C.CUSTOMER_ID = O.CUSTOMER_ID;

View created.
```

```
CUSTOMER_ID FIRST_NAME                                        LAST_NAME                                         ORDER_I ORDER_DAT TOTAL_AMOUNT
----------- ------------------------------------------------- ------------------------------------------------- ------- --------- ------------
ORDER_STATUS
--------------------
          1 Arun                                              Kumar                                             0001    28-SEP-26         2800
DELIVERED

          2 Meena                                             Ravi                                              0002    25-SEP-26         1500
SHIPPED

          3 Karthik                                           Raj                                               0003    04-OCT-26         3000
PENDING

          4 Swetha                                            Mohan                                             0004    07-OCT-26         4000
DELIVERED

          5 Imran                                             Ali                                               0005    12-OCT-26         1000
DELIVERED

          6 Keerthi                                           Suresh                                            0006    18-OCT-26         1200
PENDING

          7 Rohan                                             Sharma                                            0007    20-OCT-26         5000
SHIPPED

          8 Pooja                                             Nair                                              0008    25-OCT-26          800
SHIPPED

          9 Sanjay                                            Das                                               0009    30-OCT-26          300
DELIVERED

         10 Harini                                            Krishnan                                          0010    01-NOV-26          500
PENDING


10 rows selected.
```

2. Product Sales Report View

Display:

- Product Name
- Quantity Sold
- Total Revenue

```
SQL> CREATE OR REPLACE VIEW PRODUCT_SALES_REPORT AS
  2  SELECT
  3      P.PRODUCT_ID,
  4      P.PRODUCT_NAME,
  5      SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  6      SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  7  FROM PRODUCT P
  8  JOIN ORDER_DETAILS OD
  9  ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME;

View created.

SQL> SELECT * FROM PRODUCT_SALES_REPORT;

PRODUCT_ID PRODUCT_NAME         QUANTITY_SOLD TOTAL_REVENUE
---------- -------------------- ------------- -------------
       101 Laptop                          11        605000
       102 Smart TV                        12         15030
```

3. Payment Status View

Display:

- Order ID
- Payment Mode
- Payment Status
- Transaction Amount

```
SQL> CREATE OR REPLACE VIEW PAYMENT_STATUS_REPORT AS
  2  SELECT
  3      ORDER_ID,
  4      PAYMENT_MODE,
  5      PAYMENT_STATUS,
  6      TRANSACTION_AMOUNT
  7  FROM PAYMENT;

View created.

SQL> SELECT * FROM PAYMENT_STATUS_REPORT;

  ORDER_ID PAYMENT_MODE         PAYMENT_STATUS       TRANSACTION_AMOUNT
---------- -------------------- -------------------- ------------------
         1 UPI                  Success                            1500
         2 Card                 Success                            2500
         3 UPI                  Failed                             1200
         4 Net Banking          Success                            3000
         5 Card                 Failed                             1800
         6 Cash                 Pending                             800
         7 UPI                  Success                            2200
         8 Card                 Success                            4500
         9 Net Banking          Failed                             1700
        10 UPI                  Success                            3200
      1011 Cash                 Success                             900
      1012 Card                 Pending                            2800
      1013 UPI                  Failed                             1100
      1014 Net Banking          Success                            5000
      1015 Card                 Success                            3500

15 rows selected.
```

2. Develop Stored Procedures for Order Processing

1. Add New Customer Order

Process:

- Insert order details.
- Add purchased products.
- Calculate total amount.

```
SQL> CREATE OR REPLACE PROCEDURE ADD_CUSTOMER_ORDER
  2  (
  3      P_ORDER_ID IN VARCHAR2,
  4      P_CUSTOMER_ID IN NUMBER,
  5      P_ORDER_DETAIL_ID IN VARCHAR2,
  6      P_PRODUCT_ID IN NUMBER,
  7      P_QUANTITY IN NUMBER,
  8      P_PAYMENT_ID IN NUMBER,
  9      P_PAYMENT_MODE IN VARCHAR2
 10  )
 11  AS
 12      V_PRICE NUMBER(10,2);
 13      V_STOCK NUMBER(5);
 14      V_TOTAL NUMBER(10,2);
 15  BEGIN
 16      SELECT PRICE, STOCK_QUANTITY
 17      INTO V_PRICE, V_STOCK
 18      FROM PRODUCT
 19      WHERE PRODUCT_ID = P_PRODUCT_ID;
 20
 21      IF P_QUANTITY <= 0 THEN
 22          RAISE_APPLICATION_ERROR(-20001, 'Invalid quantity');
 23      END IF;
 24
 25      IF V_STOCK < P_QUANTITY THEN
 26          RAISE_APPLICATION_ERROR(-20002, 'Insufficient stock');
 27      END IF;
 28
 29      V_TOTAL := V_PRICE * P_QUANTITY;
 30
 31      INSERT INTO ORDERS
 32      (ORDER_ID, CUSTOMER_ID, ORDER_DATE,
 33       TOTAL_AMOUNT, ORDER_STATUS)
 34      VALUES
 35      (P_ORDER_ID, P_CUSTOMER_ID, SYSDATE,
 36       V_TOTAL, 'Pending');
 37
 38      INSERT INTO ORDER_DETAILS
 39      (ORDER_DETAIL_ID, ORDER_ID, PRODUCT_ID,
 40       QUANTITY, PRICE)
 41      VALUES
 42      (P_ORDER_DETAIL_ID, P_ORDER_ID,
 43       TO_CHAR(P_PRODUCT_ID), P_QUANTITY, V_PRICE);
 44
 45      UPDATE PRODUCT
 46      SET STOCK_QUANTITY = STOCK_QUANTITY - P_QUANTITY
 47      WHERE PRODUCT_ID = P_PRODUCT_ID;
 48
 49      INSERT INTO PAYMENT
 50      (PAYMENT_ID, ORDER_ID, PAYMENT_DATE,
 51       PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)
 52      VALUES
 53      (P_PAYMENT_ID, P_ORDER_ID, SYSDATE,
 54       P_PAYMENT_MODE, 'Pending', V_TOTAL);
 55
 56      DBMS_OUTPUT.PUT_LINE('Order Added Successfully');
 57      DBMS_OUTPUT.PUT_LINE('Total Amount: ' || V_TOTAL);
 58
 59  EXCEPTION
 60      WHEN NO_DATA_FOUND THEN
 61          RAISE_APPLICATION_ERROR(-20003, 'Product not found');
 62  END;
 63  /

Procedure created.
```

2. Update Order Status

Example:

Change: Pending → Shipped → Delivered

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_ORDER_STATUS
  2  (
  3      P_ORDER_ID IN VARCHAR2,
  4      P_STATUS IN VARCHAR2
  5  )
  6  AS
  7  BEGIN
  8      UPDATE ORDERS
  9      SET ORDER_STATUS = P_STATUS
 10      WHERE ORDER_ID = P_ORDER_ID;
 11
 12      IF SQL%ROWCOUNT = 0 THEN
 13          DBMS_OUTPUT.PUT_LINE('Order Not Found');
 14      ELSE
 15          DBMS_OUTPUT.PUT_LINE('Order Status Updated Successfully');
 16      END IF;
 17  END;
 18  /

Procedure created.
```

```
SQL> EXEC UPDATE_ORDER_STATUS('O2001', 'Shipped');
Order Not Found

PL/SQL procedure successfully completed.

SQL> EXEC UPDATE_ORDER_STATUS('O2001', 'Delivered');
Order Not Found

PL/SQL procedure successfully completed.

SQL> SELECT ORDER_ID, ORDER_STATUS
  2  FROM ORDERS;

ORDER_I ORDER_STATUS
------- --------------------
0001    DELIVERED
0002    SHIPPED
0003    PENDING
0004    DELIVERED
0005    DELIVERED
0006    PENDING
0007    SHIPPED
0008    SHIPPED
0009    DELIVERED
0010    PENDING

10 rows selected.
```

3. Generate Customer Purchase History

Input:

Customer_ID

Output:

- Order details
- Purchase amount
- Order status

```
SQL> CREATE OR REPLACE PROCEDURE CUSTOMER_PURCHASE_HISTORY
  2  (
  3      P_CUSTOMER_ID IN NUMBER
  4  )
  5  AS
  6  BEGIN
  7      FOR R IN
  8      (
  9          SELECT ORDER_ID,
 10                 ORDER_DATE,
 11                 TOTAL_AMOUNT,
 12                 ORDER_STATUS
 13          FROM ORDERS
 14          WHERE CUSTOMER_ID = P_CUSTOMER_ID
 15      )
 16      LOOP
 17          DBMS_OUTPUT.PUT_LINE(
 18              'Order ID: ' || R.ORDER_ID
 19          );
 20
 21          DBMS_OUTPUT.PUT_LINE(
 22              'Order Date: ' || R.ORDER_DATE
 23          );
 24
 25          DBMS_OUTPUT.PUT_LINE(
 26              'Purchase Amount: ' || R.TOTAL_AMOUNT
 27          );
 28
 29          DBMS_OUTPUT.PUT_LINE(
 30              'Order Status: ' || R.ORDER_STATUS
 31          );
 32      END LOOP;
 33  END;
 34  /

Procedure created.
```

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC CUSTOMER_PURCHASE_HISTORY(101)

PL/SQL procedure successfully completed.
```

3. Create Functions for Sales Calculations

1. Calculate Total Sales

Input:

- Date range

Output:

- Total revenue generated

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_TOTAL_SALES
  2  (
  3      P_START_DATE IN DATE,
  4      P_END_DATE IN DATE
  5  )
  6  RETURN NUMBER
  7  AS
  8      V_TOTAL NUMBER;
  9  BEGIN
 10      SELECT NVL(SUM(TOTAL_AMOUNT), 0)
 11      INTO V_TOTAL
 12      FROM ORDERS
 13      WHERE ORDER_DATE >= TRUNC(P_START_DATE)
 14      AND ORDER_DATE < TRUNC(P_END_DATE) + 1;
 15
 16      RETURN V_TOTAL;
 17  END;
 18  /

Function created.

SQL> SELECT CALCULATE_TOTAL_SALES
  2  (TO_DATE('01-11-2026','DD-MM-YYYY'),
  3   TO_DATE('30-11-2026','DD-MM-YYYY'))
  4  AS TOTAL_SALES
  5  FROM DUAL;

TOTAL_SALES
-----------
        500
```

2. Calculate Customer Total Spending

Input:

- Customer_ID

Output:

- Total amount spent

```
SQL> CREATE OR REPLACE FUNCTION CUSTOMER_TOTAL_SPENDING
  2  (
  3      P_CUSTOMER_ID IN NUMBER
  4  )
  5  RETURN NUMBER
  6  AS
  7      V_TOTAL NUMBER;
  8  BEGIN
  9      SELECT NVL(SUM(TOTAL_AMOUNT), 0)
 10      INTO V_TOTAL
 11      FROM ORDERS
 12      WHERE CUSTOMER_ID = P_CUSTOMER_ID;
 13
 14      RETURN V_TOTAL;
 15  END;
 16  /

Function created.

SQL> SELECT CUSTOMER_TOTAL_SPENDING(101)
  2  AS TOTAL_SPENDING
  3  FROM DUAL;

TOTAL_SPENDING
--------------
             0
```

3. Calculate Product Revenue

Input:

- Product_ID

Output:

- Total sales amount

```
SQL> CREATE OR REPLACE FUNCTION PRODUCT_REVENUE
  2  (
  3      P_PRODUCT_ID IN NUMBER
  4  )
  5  RETURN NUMBER
  6  AS
  7      V_REVENUE NUMBER;
  8  BEGIN
  9      SELECT NVL(SUM(QUANTITY * PRICE), 0)
 10      INTO V_REVENUE
 11      FROM ORDER_DETAILS
 12      WHERE PRODUCT_ID = TO_CHAR(P_PRODUCT_ID);
 13
 14      RETURN V_REVENUE;
 15  END;
 16  /

Function created.

SQL> SELECT PRODUCT_REVENUE(1)
  2  AS TOTAL_REVENUE
  3  FROM DUAL;

TOTAL_REVENUE
-------------
            0
```

4. Automate Repetitive Database Operations

Order Processing Automation

When a customer places an order:

1. Insert order details.
2. Reduce product stock.
3. Create payment record.
4. Update order status.

```
SQL> EXEC UPDATE_ORDER_STATUS('O2001', 'Shipped');
Order Not Found

PL/SQL procedure successfully completed.

SQL> SELECT * FROM ORDERS;

ORDER_I CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------- ----------- --------- ------------ --------------------
0001              1 28-SEP-26         2800 DELIVERED
0002              2 25-SEP-26         1500 SHIPPED
0003              3 04-OCT-26         3000 PENDING
0004              4 07-OCT-26         4000 DELIVERED
0005              5 12-OCT-26         1000 DELIVERED
0006              6 18-OCT-26         1200 PENDING
0007              7 20-OCT-26         5000 SHIPPED
0008              8 25-OCT-26          800 SHIPPED
0009              9 30-OCT-26          300 DELIVERED
0010             10 01-NOV-26          500 PENDING

10 rows selected.

SQL>
SQL> SELECT * FROM ORDER_DETAILS;

ORDER_DETA ORDER_I PRODUCT_ID   QUANTITY      PRICE
---------- ------- ---------- ---------- ----------
0002       0002    102                 2         15
0003       0002    101                 2      55000
0004       0002    102                 3       1500
0005       0002    101                 1      55000
0006       0002    102                 4       1500
0007       0002    101                 3      55000
0008       0002    102                 2       1500
0009       0002    101                 5      55000
0010       0002    102                 1       1500

9 rows selected.
```

```
SQL> SELECT * FROM PAYMENT;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS       TRANSACTION_AMOUNT
---------- ---------- --------- -------------------- -------------------- ------------------
       101          1 01-OCT-26 UPI                  Success                            1500
       102          2 02-OCT-26 Card                 Success                            2500
       103          3 03-OCT-26 UPI                  Failed                             1200
       104          4 04-OCT-26 Net Banking          Success                            3000
       105          5 05-OCT-26 Card                 Failed                             1800
       106          6 06-OCT-26 Cash                 Pending                             800
       107          7 07-OCT-26 UPI                  Success                            2200
       108          8 08-OCT-26 Card                 Success                            4500
       109          9 09-OCT-26 Net Banking          Failed                             1700
       110         10 10-OCT-26 UPI                  Success                            3200
       111       1011 11-OCT-26 Cash                 Success                             900
       112       1012 12-OCT-26 Card                 Pending                            2800
       113       1013 13-OCT-26 UPI                  Failed                             1100
       114       1014 14-OCT-26 Net Banking          Success                            5000
       115       1015 15-OCT-26 Card                 Success                            3500

15 rows selected.
```

Inventory Update Automation

Automatically:

- Increase stock after new supply.

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_PRODUCT_STOCK
  2  (
  3      P_PRODUCT_ID IN NUMBER,
  4      P_QUANTITY IN NUMBER
  5  )
  6  AS
  7  BEGIN
  8      UPDATE PRODUCT
  9      SET STOCK_QUANTITY = STOCK_QUANTITY + P_QUANTITY
 10      WHERE PRODUCT_ID = P_PRODUCT_ID;
 11
 12      IF SQL%ROWCOUNT = 0 THEN
 13          DBMS_OUTPUT.PUT_LINE('Product Not Found');
 14      ELSE
 15          DBMS_OUTPUT.PUT_LINE('Stock Updated Successfully');
 16      END IF;
 17  END;
 18  /

Procedure created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC UPDATE_PRODUCT_STOCK(1, 10);
Product Not Found

PL/SQL procedure successfully completed.
```

```
SQL> EXEC UPDATE_PRODUCT_STOCK(1, 10);
Product Not Found

PL/SQL procedure successfully completed.

SQL> SELECT PRODUCT_ID, PRODUCT_NAME, STOCK_QUANTITY
  2  FROM PRODUCT;

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
       101 Laptop                           10
       102 Smart TV                         15
       103 Mobile                           20
       104 Keyboard                         30
       105 Mouse                            40
       106 Headphones                       25
       107 Refrigerator                     12
       108 Washing Machine                   8
       109 Air Conditioner                  10
       110 Tablet                           18
       111 Printer                          15
       112 Smart Watch                      30
       113 Microwave                        14
       114 Camera                            7
       115 Speaker                          22
       116 Monitor                          16
       117 Power Bank                       35
       118 Gaming Laptop                     5
       119 Dishwasher                        6
       120 Electric Kettle                  28

20 rows selected.
```

- Reduce stock after purchase.

```
SQL> UPDATE PRODUCT
  2  SET STOCK_QUANTITY = STOCK_QUANTITY - 2
  3  WHERE PRODUCT_ID = 1;

0 rows updated.
```

● Identify low-stock products.

```
SQL> CREATE OR REPLACE VIEW LOW_STOCK_PRODUCTS AS
  2  SELECT
  3      PRODUCT_ID,
  4      PRODUCT_NAME,
  5      STOCK_QUANTITY
  6  FROM PRODUCT
  7  WHERE STOCK_QUANTITY < 10;

View created.

SQL> SELECT * FROM LOW_STOCK_PRODUCTS;

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
       108 Washing Machine                   8
       114 Camera                            7
       118 Gaming Laptop                     5
       119 Dishwasher                        6
```

Report Generation Automation

Automatically generate:

● Daily sales report.

```
SQL> SELECT
  2      TRUNC(ORDER_DATE) AS SALES_DATE,
  3      SUM(TOTAL_AMOUNT) AS DAILY_SALES
  4  FROM ORDERS
  5  GROUP BY TRUNC(ORDER_DATE)
  6  ORDER BY SALES_DATE;

SALES_DAT DAILY_SALES
--------- -----------
25-SEP-26        1500
28-SEP-26        2800
04-OCT-26        3000
07-OCT-26        4000
12-OCT-26        1000
18-OCT-26        1200
20-OCT-26        5000
25-OCT-26         800
30-OCT-26         300
01-NOV-26         500

10 rows selected.
```

- Monthly revenue report.

```
SQL> SELECT
  2      TO_CHAR(TRUNC(ORDER_DATE, 'MM'), 'MM-YYYY') AS SALES_MONTH,
  3      SUM(TOTAL_AMOUNT) AS MONTHLY_REVENUE
  4  FROM ORDERS
  5  GROUP BY TRUNC(ORDER_DATE, 'MM')
  6  ORDER BY TRUNC(ORDER_DATE, 'MM');

SALES_M MONTHLY_REVENUE
------- ---------------
09-2026            4300
10-2026           15300
11-2026             500
```

- Customer purchase report.

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME,
  4      C.LAST_NAME,
  5      COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  6      SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  7  FROM CUSTOMER C
  8  JOIN ORDERS O
  9  ON C.CUSTOMER_ID = O.CUSTOMER_ID
 10  GROUP BY
 11      C.CUSTOMER_ID,
 12      C.FIRST_NAME,
 13      C.LAST_NAME;

CUSTOMER_ID FIRST_NAME                                         LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
----------- -------------------------------------------------- -------------------------------------------------- ------------ --------------
          1 Arun                                               Kumar                                                         1           2800
          3 Karthik                                            Raj                                                           1           3000
          8 Pooja                                              Nair                                                          1            800
         10 Harini                                             Krishnan                                                      1            500
          6 Keerthi                                            Suresh                                                        1           1200
          9 Sanjay                                             Das                                                           1            300
          4 Swetha                                             Mohan                                                         1           4000
          5 Imran                                              Ali                                                           1           1000
          7 Rohan                                              Sharma                                                        1           5000
          2 Meena                                              Ravi                                                          1           1500

10 rows selected.
```

5. Test Database Objects

Test Views

```
SQL> SELECT * FROM CUSTOMER_ORDER_REPORT;

CUSTOMER_ID FIRST_NAME                                         LAST_NAME                                          ORDER_I ORDER_DAT TOTAL_AMOUNT
----------- -------------------------------------------------- -------------------------------------------------- ------- --------- ------------
ORDER_STATUS
--------------------
          1 Arun                                               Kumar                                              0001    28-SEP-26         2800
DELIVERED

          2 Meena                                              Ravi                                               0002    25-SEP-26         1500
SHIPPED

          3 Karthik                                            Raj                                                0003    04-OCT-26         3000
PENDING

          4 Swetha                                             Mohan                                              0004    07-OCT-26         4000
DELIVERED

          5 Imran                                              Ali                                                0005    12-OCT-26         1000
DELIVERED

          6 Keerthi                                            Suresh                                             0006    18-OCT-26         1200
PENDING

          7 Rohan                                              Sharma                                             0007    20-OCT-26         5000
SHIPPED

          8 Pooja                                              Nair                                               0008    25-OCT-26          800
SHIPPED

          9 Sanjay                                             Das                                                0009    30-OCT-26          300
DELIVERED

         10 Harini                                             Krishnan                                           0010    01-NOV-26          500
PENDING


10 rows selected.
```

```
SQL> SELECT * FROM PRODUCT_SALES_REPORT;

PRODUCT_ID PRODUCT_NAME         QUANTITY_SOLD TOTAL_REVENUE
---------- -------------------- ------------- -------------
       101 Laptop                         11        605000
       102 Smart TV                       12         15030
```

```
SQL> SELECT * FROM PAYMENT_STATUS_REPORT;

  ORDER_ID PAYMENT_MODE        PAYMENT_STATUS       TRANSACTION_AMOUNT
---------- ------------------- -------------------- ------------------
         1 UPI                 Success                            1500
         2 Card                Success                            2500
         3 UPI                 Failed                             1200
         4 Net Banking         Success                            3000
         5 Card                Failed                             1800
         6 Cash                Pending                             800
         7 UPI                 Success                            2200
         8 Card                Success                            4500
         9 Net Banking         Failed                             1700
        10 UPI                 Success                            3200
      1011 Cash                Success                             900
      1012 Card                Pending                            2800
      1013 UPI                 Failed                             1100
      1014 Net Banking         Success                            5000
      1015 Card                Success                            3500

15 rows selected.
```

```
SQL> SELECT * FROM LOW_STOCK_PRODUCTS;

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
       108 Washing Machine                   8
       114 Camera                            7
       118 Gaming Laptop                     5
       119 Dishwasher                        6
```

Test Procedures

1. Test Add Customer Order

2. Test Update Order Status

```
SQL> EXEC UPDATE_ORDER_STATUS('O2002', 'Shipped');
Order Not Found

PL/SQL procedure successfully completed.
```

3. Test Customer Purchase History

```
SQL> EXEC CUSTOMER_PURCHASE_HISTORY(101);

PL/SQL procedure successfully completed.

SQL> COMMIT;
```

Test Functions

1. Total Sales

```
SQL> SELECT CALCULATE_TOTAL_SALES
  2  (TO_DATE('01-11-2026','DD-MM-YYYY'),
  3   TO_DATE('30-11-2026','DD-MM-YYYY'))
  4  FROM DUAL;

CALCULATE_TOTAL_SALES(TO_DATE('01-11-2026','DD-MM-YYYY'),TO_DATE('30-11-2026','DD-MM-YYYY'))
-----------------------------------------------------------------------------------------------
                                                                                            500
```

2. Customer Total Spending

```
SQL> SELECT CUSTOMER_TOTAL_SPENDING(101)
  2  FROM DUAL;

CUSTOMER_TOTAL_SPENDING(101)
----------------------------
                           0
```

3. Product Revenue

```
SQL> SELECT PRODUCT_REVENUE(1)
  2  FROM DUAL;

PRODUCT_REVENUE(1)
------------------
                 0
```

Check Created Database Objects

Check Views:

```
SQL> SELECT VIEW_NAME
  2  FROM USER_VIEWS;

VIEW_NAME
------------------------------
CUSTOMER_ORDER_REPORT
LOW_STOCK_PRODUCTS
PAYMENT_STATUS_REPORT
PRODUCT_SALES_REPORT
```

Check Procedures:

```
SQL> SELECT OBJECT_NAME
  2  FROM USER_OBJECTS
  3  WHERE OBJECT_TYPE = 'PROCEDURE';

OBJECT_NAME
--------------------------------------------------------------------------------------------------------------------------------
FIRE_SUPPLIER
ADD_CUSTOMER_ORDER
UPDATE_ORDER_STATUS
CUSTOMER_PURCHASE_HISTORY
UPDATE_PRODUCT_STOCK
```

Check Functions:

```
SQL> SELECT OBJECT_NAME
  2  FROM USER_OBJECTS
  3  WHERE OBJECT_TYPE = 'FUNCTION';

OBJECT_NAME
--------------------------------------------------------------------------------------------------------------------------------
FUNCTION_NAME
CALCULATE_TOTAL_SALES
CUSTOMER_TOTAL_SPENDING
PRODUCT_REVENUE
```

Check Triggers:

```
SQL> SELECT TRIGGER_NAME
  2  FROM USER_TRIGGERS;

TRIGGER_NAME
------------------------------
PRICE_DIFFERENCE
UPDATE_BOOK_PRICE
UPDATE_PRICE
```

Check compilation errors:

```
SQL> SHOW ERRORS;
No errors.
SQL>
```